02 — BDD (Behavior-Driven Development)

Requisitos: **1** (Consultar cliente), **4** (Ativar cliente), **5** (Desativar cliente) e **8** (Consultar conta corrente).

Como foi implementado

- **Cucumber 7** com JUnit 4. Cenários escritos em **português** (`# language: pt`: `Funcionalidade`, `Cenário`, `Dado`, `Quando`, `Então`, `E`).
- Arquivos `.feature`: `test/resources/features/`
- *Step definitions* (código que executa cada passo): `test/java/bdd/`
  - `ClienteSteps.java` — requisitos 1, 4 e 5
  - `ContaSteps.java` — requisito 8 (e a transferência do doc 04)
- *Runner*: `test/java/bdd/RunCucumberTest.java`. O Maven executa esse runner e **cada cenário aparece como um teste JUnit**.
- O Cucumber cria uma instância nova das classes de steps **para cada cenário**. Assim, cada cenário começa com um `SistemaBancario` vazio e independente.

Rastreabilidade:

```
Requisito → Funcionalidade (.feature) → Cenário → Dado/Quando/Então → método Java (@Dado/@Quando/@Então)
```

Palavra-chave	Papel	Exemplo no código
Dado (Given)	preparação	`sistema.cadastrarCliente(new Cliente(...))`
Quando (When)	ação	`resultadoOperacao = sistema.ativarCliente(id)`
Então (Then)	validação	`assertEquals(SituacaoCliente.ATIVO, ...getSituacao())`

Observação honesta: os comportamentos 1, 4, 5 e 8 **já existiam** no `SistemaBancario`. Os cenários funcionam como **especificação executável** do comportamento esperado e passaram na primeira execução (`evidencias/27-bdd-cenarios-req-1-4-5-8.txt`). Nenhuma regra existente foi alterada para acomodar os cenários.

```
10 Scenarios (10 passed)
39 Steps (39 passed)
[INFO] Tests run: 10, Failures: 0, Errors: 0, Skipped: 0 -- in bdd.RunCucumberTest
```

Requisito 1 — Consultar cliente

Funcionalidade

Consulta de cliente — arquivo `test/resources/features/consultar_cliente.feature`

História

Como operador do sistema bancário **Quero** consultar um cliente pelo seu identificador **Para** visualizar os dados cadastrais do cliente

Cenário 1 — Consultar cliente existente

```
Dado que existe um cliente cadastrado com id 1 e nome "Maria Silva"
Quando o operador consultar o cliente de id 1
Então o sistema deverá retornar o cliente "Maria Silva"
```

Cenário 2 — Consultar cliente inexistente

```
Dado que não existe cliente cadastrado com id 99
Quando o operador consultar o cliente de id 99
Então o sistema deverá informar que o cliente não foi encontrado
```

Teste automatizado relacionado

- Arquivo: `test/java/bdd/ClienteSteps.java` (classe `bdd.ClienteSteps`), executado por `bdd.RunCucumberTest`

Passo	Método
Dado que existe um cliente cadastrado...	`queExisteUmClienteCadastrado(int, String)`
Dado que não existe cliente cadastrado...	`queNaoExisteClienteCadastrado(int)`
Quando o operador consultar o cliente...	`oOperadorConsultarOCliente(int)`
Então o sistema deverá retornar o cliente...	`oSistemaDeveraRetornarOCliente(String)`
Então ... cliente não foi encontrado	`oSistemaDeveraInformarClienteNaoEncontrado()`

Código de produção exercitado: `SistemaBancario.consultarCliente(int)`.

Requisito 4 — Ativar cliente

Funcionalidade

Ativação de cliente — arquivo `test/resources/features/ativar_cliente.feature`

História

Como operador do sistema bancário **Quero** ativar um cliente cadastrado **Para** permitir que ele utilize funcionalidades destinadas a clientes ativos

Cenário 1 — Ativar cliente inativo

```
Dado que existe um cliente cadastrado com id 1 e nome "Maria Silva"
E que o cliente de id 1 está inativo
Quando for solicitada a ativação do cliente de id 1
Então a operação deverá ser concluída com sucesso
E o cliente de id 1 deverá ficar ativo
```

Cenário 2 — Ativar cliente que já está ativo

```
Dado que existe um cliente cadastrado com id 1 e nome "Maria Silva"
E que o cliente de id 1 está ativo
Quando for solicitada a ativação do cliente de id 1
Então a operação deverá ser concluída com sucesso
E o cliente de id 1 deverá ficar ativo
```

(Comportamento original: a operação é idempotente.)

Cenário 3 — Tentar ativar cliente inexistente

```
Dado que não existe cliente cadastrado com id 99
Quando for solicitada a ativação do cliente de id 99
Então a operação deverá ser rejeitada
```

Teste automatizado relacionado

- Arquivo: `test/java/bdd/ClienteSteps.java` (classe `bdd.ClienteSteps`)

Passo	Método
E que o cliente de id … está inativo / ativo	`queOClienteEstaInativo(int)` / `queOClienteEstaAtivo(int)`
Quando for solicitada a ativação...	`forSolicitadaAAtivacao(int)`
Então a operação deverá ser concluída com sucesso	`aOperacaoDeveraSerConcluidaComSucesso()`
Então a operação deverá ser rejeitada	`aOperacaoDeveraSerRejeitada()`
E o cliente … deverá ficar ativo	`oClienteDeveraFicarAtivo(int)`

Código de produção exercitado: `SistemaBancario.ativarCliente(int)` → `Cliente.ativar()`.

Requisito 5 — Desativar cliente

Funcionalidade

Desativação de cliente — arquivo `test/resources/features/desativar_cliente.feature`

História

Como operador do sistema bancário **Quero** desativar um cliente cadastrado **Para** impedir que ele seja tratado como cliente ativo do banco

Cenário 1 — Desativar cliente ativo

```
Dado que existe um cliente cadastrado com id 1 e nome "Maria Silva"
E que o cliente de id 1 está ativo
Quando for solicitada a desativação do cliente de id 1
Então a operação deverá ser concluída com sucesso
E o cliente de id 1 deverá ficar inativo
```

Cenário 2 — Desativar cliente que já está inativo

```
Dado que existe um cliente cadastrado com id 1 e nome "Maria Silva"
E que o cliente de id 1 está inativo
Quando for solicitada a desativação do cliente de id 1
Então a operação deverá ser concluída com sucesso
E o cliente de id 1 deverá ficar inativo
```

Cenário 3 — Tentar desativar cliente inexistente

```
Dado que não existe cliente cadastrado com id 99
Quando for solicitada a desativação do cliente de id 99
Então a operação deverá ser rejeitada
```

Teste automatizado relacionado

- Arquivo: `test/java/bdd/ClienteSteps.java` (classe `bdd.ClienteSteps`)

Passo	Método
Quando for solicitada a desativação...	`forSolicitadaADesativacao(int)`
E o cliente ... deverá ficar inativo	`oClienteDeveraFicarInativo(int)`
(demais passos compartilhados com o requisito 4)	`queOClienteEstaAtivo`, `queOClienteEstaInativo`, `aOperacaoDeveraSer...`

Código de produção exercitado: `SistemaBancario.desativarCliente(int)` → `Cliente.desativar()`.

Requisito 8 — Consultar conta corrente

Funcionalidade

Consulta de conta corrente — arquivo `test/resources/features/consultar_conta.feature`

História

Como operador do sistema bancário **Quero** consultar uma conta corrente pelo seu número **Para** verificar o saldo e a situação da conta

Cenário 1 — Consultar conta corrente ativa

```
Dado que existe uma conta corrente cadastrada com id 1 e saldo 100,00
Quando o operador consultar a conta de id 1
Então o sistema deverá retornar a conta de id 1
E o saldo exibido deverá ser 100,00
E a situação exibida deverá ser "Ativa"
```

Cenário 2 — Consultar conta corrente inativa

```
Dado que existe uma conta corrente cadastrada com id 2 e saldo 50,00
E que a conta de id 2 está inativa
Quando o operador consultar a conta de id 2
Então o sistema deverá retornar a conta de id 2
E o saldo exibido deverá ser 50,00
E a situação exibida deverá ser "Inativa"
```

(A verificação da situação foi incluída após a auditoria: a história promete "saldo **e situação**". Evidências 43 e 44.)

Cenário 3 — Consultar conta corrente inexistente

```
Dado que não existe conta corrente cadastrada com id 99
Quando o operador consultar a conta de id 99
Então o sistema deverá informar que a conta não foi encontrada
```

Teste automatizado relacionado

- Arquivo: `test/java/bdd/ContaSteps.java` (classe `bdd.ContaSteps`)

Passo	Método
Dado que existe uma conta corrente cadastrada…	`queExisteUmaContaCadastrada(int, double)`
Dado que não existe conta corrente cadastrada…	`queNaoExisteContaCadastrada(int)`
Quando o operador consultar a conta…	`oOperadorConsultarAConta(int)`
Então o sistema deverá retornar a conta…	`oSistemaDeveraRetornarAConta(int)`
E o saldo exibido deverá ser…	`oSaldoExibidoDeveraSer(double)`
E que a conta de id … está inativa	`queAContaEstaInativa(int)`
E a situação exibida deverá ser…	`aSituacaoExibidaDeveraSer(String)`
Então … conta não foi encontrada	`oSistemaDeveraInformarContaNaoEncontrada()`

Código de produção exercitado: `SistemaBancario.consultarConta(int)`.

Nota técnica: como os cenários estão em `pt`, o Cucumber lê números no formato brasileiro (`100,00`). Foi conferido que `100,00` é lido como `100.0` e não como `10000`.

Depois do DDD

Após a modelagem de domínio (doc 03), os passos passaram a usar a linguagem do domínio: `cliente.ativar()`/`desativar()` no *Dado* e `getSituacao() == SituacaoCliente.ATIVO/INATIVO` no *Então*. Todos os cenários continuaram passando (ver `evidencias/32-suite-final-completa.txt` e, após a auditoria, `evidencias/45-suite-final-apos-auditoria.txt`: 15 cenários, 70 passos).